

Editorial — XIV Maratona UnB de Programação

25 de setembro de 2026

Problemas deste editorial

Problema	Nome
A	Árvore Six Seven
B	Bom Balanço de Divisores
C	Campinho
D	Desvendando as Formigas
E	Encontrando Ocorrências
F	Fada Quadrada
G	Grande Desastre
H	Hugo e o Jogo das Palavras
I	Igualzim, não!
J	Jogo da Memória
K	Kolisão
L	Longe de Casa
M	Malha de Amplificadores
N	Nome de Usuário

Problema A — Árvore Six Seven

O caso de borda em que a árvore possui apenas 1 vértice satisfaz a condição do problema. Se a árvore possui mais de 1 vértice, é claro que a árvore deve ter ao menos 7 vértices, do contrário é impossível satisfazer a condição.

É possível provar que as únicas construções que satisfazem a condição do problema são: -Uma “estrela” em que cada “braço” é formado por 3 ou 4 vértices (pode haver no máximo um “braço” com 4 vértices). -Duas “estrelas” em que cada “braço” é formado por 3 vértices e uma aresta ligando seus centros. Em ambos os casos, não é possível que a árvore tenha uma quantidade de vértices múltipla de 3.

Para construir (primeira construção mencionada) uma árvore nos casos que $N \geq 7$ e N não é múltiplo de 3, basta ligar cada vértice $u > 1$ ao vértice $\max(1, u - k)$ onde k é o piso de $N/3$.

Para provar o que foi afirmado, considere o vértice que separa 3 folhas em componetes diferentes e perceba que este deve ser o centro da estrela na primeira construção, ou um dos centros na segunda construção.

Problema B — Bom Balanço de Divisores

A definição de um número com bom balanço de divisores é equivalente à definição de números perfeitos: números n cuja soma dos divisores próprios é n . O problema pede que se conte números perfeitos até N .

Podemos usar o fato de que existe uma bijeção entre primos de Mersenne e números perfeitos pares (não sabemos se existem números perfeitos ímpares, mas, se existirem, eles são muito maiores que 2^{63}). O i -ésimo número perfeito par, digamos n , está associado ao i -ésimo primo de Mersenne, digamos $2^x - 1$, pela fórmula: $n = (2^x - 1) \cdot 2^{x-1}$. Isso implica que todos os números perfeitos pares têm a forma: $11 \dots 110 \dots 00$, com x dígitos 1 à esquerda e $x-1$ dígitos 0 à direita. Podemos ver que existem, no máximo, 32 números perfeitos até 2^{63} , então podemos simplesmente encontrá-los, listá-los e responder trivialmente à contagem para a entrada N .

Listar os números perfeitos é uma tarefa fácil de se fazer no seu próprio computador. Você pode simplesmente iterar por todos os números n da forma binária descrita (candidatos a números perfeitos) e verificar a condição da soma dos divisores localmente. Sendo um teste local, é possível encontrar os divisores dos candidatos n em $O(\sqrt{n})$ – mesmo lento demais para submeter, ainda é rápido para execuções no PC ao longo da prova. Assim, você pode encontrá-los e inseri-los diretamente no código da sua solução.

No entanto, também é possível resolver o problema dentro da solução submetida. Pode-se usar a mesma abordagem de verificação da soma dos divisores para cada candidato, mas calculando a soma dos divisores a partir de uma fórmula baseada na fatoração do número, que pode ser obtida por algoritmos como Pollard's Rho. Alternativamente, você também pode apenas verificar a primalidade do número formado pelo prefixo de 1s de cada candidato. Como esses números vão até 2^{32} , uma verificação simples com iteração até a raiz é rápida o suficiente para verificar os 32 candidatos.

Problema C — Campinho

Como todos os preços e pontuações têm exatamente duas casas decimais, podemos multiplicar todos eles por 100 e trabalhar apenas com inteiros. Isso significa que o orçamento máximo de João (X) passa a ser 20000.

Podemos resolver o problema usando programação dinâmica (DP).

Primeiro, podemos separar os jogadores de acordo com suas posições. Processaremos as posições na ordem G, Z, L, M e A. Para cada posição, sabemos exatamente quantos jogadores devem ser escolhidos: 1, 2, 2, 3 e 3, respectivamente.

Seja $dp[c][i][k][x]$ a maior pontuação adicional que podemos obter se estivermos processando a posição c , considerando o jogador i dessa posição, tendo já escolhido k jogadores dessa posição e tendo gasto x no total.

Para cada jogador, há duas possibilidades: comprá-lo ou ignorá-lo. Se ignorarmos o jogador atual, passamos para o próximo jogador sem alterar o número de jogadores selecionados nem o valor gasto. Se comprarmos o jogador atual, com preço P_i e pontuação S_i , aumentamos o número de jogadores selecionados em 1, o valor gasto em P_i e a pontuação em S_i .

Portanto, a transição é

$$dp[c][i][k][x] = \max \left\{ \begin{array}{l} dp[c][i+1][k][x], \\ S_i + dp[c][i+1][k+1][x+P_i]. \end{array} \right.$$

Um estado é inválido se mais jogadores do que o necessário para a posição atual tiverem sido escolhidos, ou se o valor total gasto for maior que X .

Quando todos os jogadores da posição atual tiverem sido considerados, podemos passar para a próxima posição somente se exatamente o número exigido de jogadores da posição atual tiver sido escolhido. Nesse caso, começamos a processar a próxima posição com 0 jogadores escolhidos dela, mantendo o valor total já gasto.

Depois que todas as cinco posições tiverem sido processadas, teremos montado um time válido, então a pontuação adicional é 0. A resposta é obtida começando pela posição G, antes de considerar qualquer jogador, sem nenhum jogador escolhido e sem dinheiro gasto.

Se nenhuma sequência válida de escolhas conseguir montar a formação 4-3-3 sem gastar mais do que X , a resposta é -1 .

Há 5 posições possíveis, no máximo N jogadores a considerar, no máximo 3 jogadores selecionados para a posição atual e no máximo 20000 valores possíveis para o valor gasto. Como o número de posições e o número máximo de jogadores exigidos são constantes, a complexidade da solução é $O(NX)$.

Problema D — Desvendando as Formigas

O principal desafio do problema é o evento do tipo 2. Se movermos individualmente todas as formigas sempre que esse evento acontecer, uma única operação pode custar $O(Q)$, tornando a solução lenta demais.

Podemos evitar isso mantendo um deslocamento global.

Seja *offset* a soma de todos os deslocamentos realizados até o momento. Em vez de armazenar a posição real p de uma formiga, armazenamos

$$p - \textit{offset}.$$

Chamaremos esse valor de posição relativa da formiga.

Dessa forma, quando acontece um evento 2 x , não precisamos alterar nenhuma posição armazenada. Basta fazer

$$\textit{offset} \leftarrow \textit{offset} + x.$$

De fato, todas as formigas passam a estar x posições mais à direita, exatamente como desejado.

Agora, considere um evento 1 x . Queremos inserir uma formiga cuja posição real é x . Como armazenamos posições relativas, devemos inserir

$$x - \textit{offset}.$$

Se já existir uma formiga nessa posição, esse valor já estará no conjunto. Como queremos apenas substituir a formiga antiga, basta manter uma única cópia desse valor.

Por fim, em um evento 3 x , queremos contar quantas formigas estão em posições reais p tais que

$$p < x.$$

Como todas as formigas possuem o mesmo deslocamento global,

$$p < x$$

é equivalente a

$$p - \textit{offset} < x - \textit{offset}.$$

Portanto, basta contar quantas posições relativas armazenadas são estritamente menores que

$$x - \textit{offset}.$$

Assim, após tratar o deslocamento, o problema se reduz a manter um conjunto de números com duas operações:

Essa é exatamente a interface do que é conhecido como “Ordered Set”. Em C++, podemos usar a estrutura `tree` da biblioteca `ext/pb_ds`, que possui a operação `order_of_key(x)`, responsável por retornar a quantidade de elementos estritamente menores que x .

Portanto, para cada evento, após decodificar

$$x = v \oplus r,$$

fazemos:

- tipo 1: inserir $x - offset$;
- tipo 2: somar x a $offset$;
- tipo 3: calcular `order_of_key(x-offset)` e atualizar r com essa resposta.

Também é possível implementar as mesmas operações usando outras estruturas de dados. Por exemplo, uma Treap pode armazenar, em cada nó, o tamanho de sua subárvore, permitindo contar quantos valores são menores que um dado número. Outra possibilidade é uma Segment Tree dinâmica sobre o intervalo de posições possíveis.

Em qualquer uma dessas estruturas, o custo esperado de cada operação é $O(\log Q)$, e a complexidade total é $O(Q \log Q)$.

Problema E — Encontrando Ocorrências

A resposta para o problema é $\prod_{i=1}^N c_i + 1$, onde c_i é o número de ocorrências de K em B_i . Isso porque as escolhas de ocorrências a serem adicionadas na alucinação são independentes entre os livros.

Resta o problema de string matching: contar ocorrências de K em B_i . Há muitas maneiras conhecidas de resolver esse problema. Algumas podem levar a uma solução direta, mas consideraremos um caso em que uma otimização é necessária.

Suponha que você saiba como contar ocorrências de K em B_i em $O(|K| + |B_i|)$. Uma aplicação ingênua levaria a uma solução de $O\left(\sum_{i=1}^N |K| + |B_i|\right) = O\left(N \cdot |K| + \sum_{i=1}^N |B_i|\right)$, o que é lento demais. Mas, se você só executar o algoritmo quando $|K| \leq |B_i|$, então a complexidade fica limitada por $O\left(2 \cdot \sum_{i=1}^N |B_i|\right)$, o que passa.

Problema F — Fada Quadrada

A solução é simples: enquanto não houver o fim da entrada (EOF), leia o número dado. Se o número for um quadrado, basta apresentá-lo. Caso contrário, encontre o maior valor quadrado que seja menor que o número e o menor valor quadrado que seja maior que ele, e calcule as diferenças destes valores para o próprio número. Apresente o valor quadrado que tenha a menor destas diferenças. Note que as diferenças nunca serão iguais, porque uma será ímpar e a outra será par.

Problema G — Grande Desastre

Esse problema pode ser resolvido adaptando o conhecido algoritmo de Tarjan para detecção de pontos de articulação. Nessa adaptação, construímos o que chamamos de block-cut-tree, o que, essencialmente, é uma decomposição do grafo em componentes biconexas. A decomposição gera um grafo modificado em que:

- Cada componente biconexa corresponderá a um vértice do tipo block.
- Cada vértice de articulação corresponderá a um vértice do tipo cut.
- Cada vértice do tipo block está conectado ao vértice do tipo cut correspondente.

Note que um vértice de articulação no grafo original estará representado tanto em um vértice do tipo block quanto em um vértice do tipo cut, já que o vértice participa de uma componente biconexa.

Uma vez calculada a block-cut-tree a resposta é: $\max(d-1, \lceil l/2 \rceil)$, em que $d = \max\{\text{degree}(v) \mid v \in V\}$, onde V é o conjunto de vértices da block-cut-tree que correspondem à **vértices de articulação**, e l é o número de folhas da block-cut-tree. Esse fórmula vem dos Teoremas 6 e 7 de Eswaran e Tarjan (1976) (Augmentation Problems, SIAM J. Comp.).

Nota: durante a prova da XIV Maratona UnB de Programação, a solução modelo utilizada não calculava o termo $d-1$, portanto estava incorreta. Após a prova o erro foi detectado Eu, Daniel Saad, autor do problema, assumo todas as responsabilidades pelo erro e agradeço por apontá-lo.

Problema H — Hugo e o Jogo das Palavras

Neste problema, podemos testar todas as possibilidades de troca que Hugo pode fazer, e verificar se ele e as crianças ganhariam o jogo em alguma delas.

Para uma string de tamanho N , existem $\frac{N \cdot (N-1)}{2} = O(N^2)$ pares de posições distintas. E, para duas strings de tamanhos N e M , verificar se a primeira é menor (na ordem alfabética) que a segunda pode ser feito em complexidade $O(N+M)$, iterando posição por posição, como descrito no enunciado.

Assim, ao testar fazer a troca para todos os pares de posição de s , podemos verificar se a string resultante é menor do que t , em complexidade total $O(n^2(n+m))$. Como Hugo também pode fazer a troca em t , também testamos todas as trocas possíveis, em $O(m^2(n+m))$. Não podemos deixar de testar não fazer trocas também, em $O(n+m)$.

Vemos que no final, a complexidade de testar todas essas possibilidades é $O((n+m) \cdot (1+n^2+m^2))$. Denotando o valor máximo (neste problema, 100) de n e m por N , a complexidade é portanto $O(N^3)$, o que é suficiente.

Problema I — Igualzim, não!

Podemos representar os moradores e suas relações de compadrio como um grafo: cada morador é um vértice, e existe uma aresta entre dois vértices se os moradores correspondentes são compadres.

Como existem apenas duas cores disponíveis e dois compadres não podem ter casas da mesma cor, precisamos verificar se é possível colorir o grafo usando duas cores de forma que vértices ligados por uma aresta tenham cores diferentes. Isso é exatamente verificar se o grafo é **bipartido**.

Para cada componente conexa do grafo, escolhemos uma cor para um vértice qualquer. A partir daí, as cores de todos os outros vértices da componente ficam determinadas: cada vizinho deve receber a cor oposta.

Se em algum momento encontrarmos dois vértices adjacentes com a mesma cor, então o grafo não é bipartido e não existe nenhuma forma válida de pintar as casas. Nesse caso, a resposta é 0.

Caso contrário, cada componente conexa possui exatamente duas formas de ser colorida: podemos trocar azul por branco e branco por azul em todos os seus vértices.

Assim, se o grafo possui (C) componentes conexas e todas elas são bipartidas, a resposta é $2^C \pmod{10^9+7}$.

Podemos percorrer cada componente usando DFS ou BFS, marcando os vértices com uma das duas cores e verificando se algum conflito ocorre.

Como cada vértice e cada relação de compadrio são processados uma quantidade constante de vezes, a complexidade é $O(N + \sum m_i)$.

Problema J — Jogo da Memória

Jogo da memória — editorial

O nome do jogo entrega a estratégia: **memória**. Cada resposta DIFERENTE v_1 v_2 revela o valor de duas cartas; guarde tudo e nunca esqueça. Mas, com o limite apertado em exatamente $N \cdot M$ jogadas, lembrar não basta: é preciso **revelar duas cartas novas por jogada** e fechar cada par com uma única jogada.

Estratégia

Mantenha três coisas: o conjunto das cartas **já emparelhadas**, um mapa das cartas **já reveladas** (mas ainda sem par) e a lista das cartas **ainda desconhecidas**. A cada jogada:

1. **Se você já conhece duas cartas sem par de mesmo valor**, vire essas duas. O par fecha na hora (resposta IGUAL) sem gastar nenhuma revelação nova.
2. **Senão**, vire duas cartas **ainda desconhecidas**. Se por sorte forem iguais, o par fechou; caso contrário, você aprende os dois valores para usar na regra 1 mais tarde.

Como a regra 1 tem prioridade, todo par cujas duas cartas você já viu é resolvido imediatamente, e nenhuma carta é revelada duas vezes.

Por que cabe exatamente no limite

Há $N \cdot M = 2P$ cartas e P pares. Cada jogada de exploração (regra 2) revela **duas** cartas ainda desconhecidas, então bastam P explorações para ver o tabuleiro inteiro. Cada par precisa, no pior caso, de mais **uma** jogada para ser fechado (regra 1), a menos que suas duas cartas tenham sido reveladas juntas numa mesma exploração, o que só ajuda. No pior caso adversarial isso dá

$$P + P = 2P = N \cdot M \text{ jogadas,}$$

exatamente o limite. Como o número de cartas desconhecidas começa par e cai de 2 em 2, a regra 2 sempre encontra duas desconhecidas; o caso “sobrou só uma” nunca acontece.

Esse $N \cdot M$ é o melhor pior-caso possível para quem vira as duas cartas **de uma vez**, sem poder escolher a segunda depois de ver a primeira.

Cuidados de implementação

- **Dê flush a cada jogada** e leia a resposta antes da próxima: é um problema interativo.
- Em linguagens que leem a entrada preguiçosamente (Rust, C++ com stream), leia **uma linha por vez**; ler tudo até o EOF trava, porque o juiz só fala depois de cada jogada sua.
- Com $N \cdot M \leq 10^4$, procurar o par conhecido varrendo todas as cartas a cada jogada é $O((N \cdot M)^2)$ no total, o que passa em C mas não em Python nem em Bash. Use um mapa **valor $\rightarrow$ posição da carta solitária** (ou um vetor indexado por valor) e um ponteiro que avança pelas desconhecidas: $O(1)$ por revelação, $O(N \cdot M)$ no total.

Problema K — Kolisão

Existem algumas soluções diferentes para esse problema. Aqui descrevemos duas:

1. Primeira solução - Soma de Minkowski

Primeiramente, vamos relembrar como verificar se dois polígonos têm alguma interseção. Podemos calcular o polígono $C = A + (-B)$, onde $+$ é a soma de Minkowski e $(-B)$ é obtido de B após inverter os sinais, e verificar se a origem está dentro de C .

A soma de Minkowski pode ser definida como o conjunto de pontos $D + E = \{d + e \mid d \in D, e \in E\}$. Assim, $(0, 0)$ estar dentro de $A + (-B)$ significa que existem $a \in A$ e $b \in B$ tais que $a + (-b) = 0$ ou seja $a = b$.

Agora, precisamos ver como o deslocamento de cada polígono interfere na soma de Minkowski. Após t unidades de tempo, os polígonos resultantes serão $A_t = \{a + v_a * t \mid a \in A\}$ e $B_t = \{b + v_b * t \mid b \in B\}$.

$$\begin{aligned} A_t + (-B_t) &= \{a_t + (-b_t) \mid a \in A_t, b \in B_t\} \\ &= \{(a + v_a * t) - (b + v_b * t) \mid a \in A, b \in B\} && \text{(expandindo as definições de } A_t \text{ e } B_t) \\ &= \{a + (-b) + (v_a - v_b) * t \mid a \in A, b \in B\} && \text{(reordenando os termos)} \\ &= C + t(v_a - v_b) && \text{(definição de } C) \end{aligned}$$

Assim, temos o polígono C movendo com velocidade $v_a - v_b$ e precisamos encontrar o primeiro momento que este passa pela origem.

Para simplificar a implementação, podemos considerar a semirreta partindo da origem com direção $v_b - v_a$. Esta só pode ter tocado o polígono C em um ou dois pontos e só precisamos encontrar o mais perto da origem.

Vale lembrar que podemos determinar se há colisão ou não, ou seja, se a semirreta passa por C ou não, usando apenas inteiros. Só precisamos de ponto flutuante para recuperar a resposta.

Complexidade da solução $O(N + M)$.

2. Segunda solução - Colisão entre cada ponto e um polígono

Podemos simplificar ligeiramente o problema mudando o referencial para o polígono B e fazer A se mover com velocidade $v_a - v_b$.

Então, para cada vértice p de A , podemos encontrar o instante de colisão com B .

Esta solução cobre quase todos os casos, a menos que um vértice de B toque pela primeira vez numa aresta de A . Mas podemos inverter A com B , aplicar o mesmo algoritmo e usar a melhor das duas respostas.

2.1 Busca binária

Com uma busca binária, encontramos a “metade” do polígono virada para o lado de p . Ou seja, a parte do polígono onde é possível que p colida, independentemente da velocidade.

Caso este intervalo esteja totalmente de um lado da reta r , reta com inclinação $v_a - v_b$ que passa por p , não temos colisão.

Caso contrário, temos uma interseção e podemos usar uma segunda busca binária para encontrar a aresta exata de contato, com um vértice em cima e um vértice embaixo de r .

Na figura, as arestas em preto formam a “metade” do polígono virada para o lado de p . O vetor indica a velocidade relativa $v = v_a - v_b$, e a busca binária nesses segmentos de retas localiza a primeira aresta atingida pela trajetória de p .

Complexidade final $O((N + M) \log(N + M))$.

2.2 Varredura

Todos os pontos terão a mesma velocidade $v_a - v_b$. Então podemos fazer um *sweep line*.

É mais comum usar a técnica *sweep line* com uma reta paralela a um dos eixos se movendo, mas podemos usar qualquer reta.

Aqui usaremos uma reta com inclinação $v_a - v_b$ fazendo uma varredura pelo plano. Nela, iniciamos com nenhum ponto de interseção com o polígono, passamos a ter um ponto, depois dois pontos, depois um ponto, e por fim, nenhum ponto. Podemos manter os índices das arestas do polígono onde a reta passa.

Para ordenar os eventos, basta usar a área com sinal entre cada ponto e a reta. Esta ordenação pode ser feita com números inteiros.

Para responder uma consulta, só é preciso calcular o ponto mais perto, dentre os dois de interseção.

Complexidade final $O((N + M) \log(N + M))$.

Problema L — Longe de Casa

Considere o i -ésimo explorador, cujas posição e energia iniciais são (x_i, y_i) e k_i . Seja $e_i = k_i - 1$. Se a posição de pouso do helicóptero for (H_x, H_y) , esse explorador só conseguirá ser salvo se $H_x \in [x_i - e_i, x_i + e_i]$ ou $H_y \in [y_i - e_i, y_i + e_i]$ — a ideia é que o explorador pode usar as primeiras e_i unidades de energia para se deslocar em um sentido, e reservar a última unidade para o foguete, podendo alcançar qualquer coordenada no outro sentido.

Observe que não é possível para esse explorador ter movido mais do que e_i nos dois sentidos ao mesmo tempo. Para mover mais do que e_i em um dado sentido, ele precisa ou gastar toda a sua energia nesse sentido (movendo exatamente $k_i = e_i + 1$) ou reservar uma unidade de energia para usar o foguete (que só pode ser usado uma vez). Essa observação é suficiente para provar a afirmação do parágrafo anterior.

Para encontrar o (H_x, H_y) ótimo, considere fixar H_x . Feito isso, todos os exploradores i com $H_x \in [x_i - e_i, x_i + e_i]$ conseguem ser resgatados. Diremos que esses exploradores compõem o conjunto $SX(H_x)$.

Considere agora os exploradores restantes, que compõem o conjunto $SY(H_x) = E \setminus SX(H_x)$, sendo E o conjunto de todos os exploradores. Para eles, basta escolher H_y que maximize a quantidade dos exploradores j em $SY(H_x)$ tais que $H_y \in [y_j - e_j, y_j + e_j]$. Esse valor pode ser obtido ao calcularmos, para todo H_y possível, a quantidade de intervalos $[y_j - e_j, y_j + e_j]$ que o contém (a quantidade de H_y s que estão dentro de algum intervalo é limitada pelos valores mínimos e máximos das bordas desses intervalos — menor do que 10^6 neste problema).

Esse cálculo pode ser feito utilizando uma Segment Tree com Lazy Propagation, com update de soma em intervalo e query de máximo em intervalo. Basta somar 1 em todo intervalo $[y_j - e_j, y_j + e_j]$ para os exploradores j em $SY(H_x)$ e encontrar o máximo com uma query que contenha o intervalo todo mantido pela Segment Tree. A resposta ótima para esse H_x é, então, esse máximo da Segment Tree somado a $|SX(H_x)|$ (exploradores cujos intervalos não foram utilizados na Segment Tree).

No entanto, resolvemos o problema apenas para um H_x fixo. Para resolver o problema para todo H_x , podemos utilizar uma ideia de Sweep Line na coordenada x .

Inicialmente, fixe H_x em uma posição à esquerda o bastante para que $SX(H_x) = \emptyset$ e monte a Segment Tree como descrito — a resposta atual é o máximo na Segment Tree. Sabemos que um explorador i irá entrar em $SX(H_x)$ quando H_x atingir $x_i - e_i$ e sairá de $SX(H_x)$ quando H_x ultrapassar $x_i + e_i$. Dessa forma, podemos mover H_x progressivamente para a direita e atualizando a resposta, mantendo apenas a Segment Tree e o tamanho de $SX(H_x)$: se um explorador entra em $SX(H_x)$, removemos seu intervalo da Segment Tree; se um explorador sai de $SX(H_x)$, adicionamos seu intervalo de volta à Segment Tree. Assim como para H_y , a quantidade de H_x s que estão dentro de algum intervalo é pequena o bastante pelos limites do problema para realizar essa progressão para a direita.

Dessa forma, conseguimos calcular a resposta quando $SX(H_x)$ é vazio e também para todo $SX(H_x)$ não vazio possível, ao mover H_x progressivamente para a direita. A maior dentre essas respostas é a que deve ser imprimida como a resposta final do problema.

Note que a complexidade desse algoritmo é eficiente devido ao fato de que a quantidade de posições relevantes de H_x e H_y é limitada pelos intervalos dos exploradores. Além disso, a quantidade total de updates e queries na Segment Tree é $O(N)$, o que é eficiente para os limites do problema.

Problema M — Malha de Amplificadores

Primeiro resolvemos o problema no caso em que a árvore dada na entrada é uma reta. Para isso, montamos uma árvore de segmentos lazy em que cada vértice guarda um par (a, b) , que representa a função afim $f(x) = ax + b$ obtida pela composição das funções do seu intervalo.

Notemos que a composição de funções afins é afim, e que a composição de funções é associativa. Além disso, podemos calcular a composição da função $f(x) = cx + d$ com ela mesma L vezes em tempo $O(\log L)$ utilizando exponenciação rápida.

Isso é suficiente para construir uma árvore de segmentos lazy que resolve a versão do problema em que a árvore dada é uma reta em $O(N + Q \log^2 N)$. Ao atribuir uma função a todos os elementos de um intervalo, utilizamos a exponenciação rápida para calcular a composição correspondente a cada segmento afetado.

Utilizando Heavy-Light decomposition podemos transformar qualquer solução para um problema de consultas em intervalos em uma reta em uma solução para o problema análogo em uma árvore, à custo de um fator a mais de $\log(N)$ (veja por exemplo <https://github.com/e-so-fazer/kactl/blob/main/content/graph/HLD.h>). Assim, a complexidade final é $O(N + Q \log^3 N)$.

Também é possível otimizar a versão na reta para $O(N + Q \log N)$ e, portanto, obter a complexidade final $O(N + Q \log^2 N)$, mas isso não é necessário dadas as restrições de tempo do problema. Para isso, antes de cada atualização, calculamos uma única vez as composições da função com ela mesma $1, 2, 4, 8, \dots$ vezes, em tempo $O(\log N)$. Utilizando uma árvore de segmentos cujo tamanho é uma potência de dois, cada segmento corresponde a uma dessas quantidades. As marcações lazy compartilham essa tabela, permitindo aplicar e propagar a atualização em tempo constante por vértice, sem repetir a exponenciação.

Problema N — Nome de Usuário

O problema é uma aplicação direta de manipulação de strings.

Deve-se ler a entrada e primeiro verificar se atende à restrição de tamanho imposta no enunciado. Se não atender, deve-se imprimir a mensagem pedida.

Caso atenda, basta realizar as substituições de caracteres pedidas pelo enunciado e imprimir o resultado.